

Design and Implementation of an Arbitrary Precision Arithmetic Library in Java

Author:

Rishit Mittal

CS24BTECH11053

May 1, 2025

Abstract

This report presents the design rationale, implementation strategy, verification approach, and key insights gained during the development of a Java-based library for arbitrary precision arithmetic. The library enables mathematical operations on integers and floating-point numbers with theoretically unbounded precision, making it suitable for scientific and cryptographic computations requiring high accuracy.

Contents

1	Introduction	2
2	Features and Language Extensions	2
3	User Guide (README Summary)	6
4	Git Commit History Snapshot	7
5	Limitations	8
6	Verification and Testing Approach	8
7	Key Learnings	9

1 Introduction

This project aims to implement a high-precision arithmetic library capable of operating on both integers and floating-point numbers with arbitrary digit length. It was developed with object-oriented programming principles and is compatible with both Java and Python interfaces. The motivation behind this project stems from the limitations of built-in data types in traditional programming languages which cannot natively support operations on extremely large or precise numerical values.

2 Features and Language Extensions

This section outlines the major features provided by the library and presents illustrative examples for each. These enhancements effectively extend the capabilities of the Java language in terms of numeric computation.

Key Functionalities

- Support for arbitrary-length integer arithmetic via the `AInteger` class
- Floating-point arithmetic with up to 1000 digits after the decimal point using `AFloat`
- Support for basic arithmetic operations: addition, subtraction, multiplication, and division
- Operator overloading through intuitive method interfaces (e.g., `add()`, `sub()`, etc.)
- Dual CLI support through both Java and Python scripts
- Modular and reusable packaging using `.jar`, with Maven integration

Illustrative Examples

Integer Addition:

```
AInteger a = new AIInteger("1234");
AInteger b = new AIInteger("4321");
AInteger result = a.add(b); // Output: 5555
System.out.println(result);
```

Floating-point Multiplication (via Python CLI):

```
$ python run.py float mul 1.5 2.5
# Output: 3.75
```

Design Decisions

AInteger

The `AInteger` class provides support for arbitrary-precision integers using a list-based digit representation and handles all basic integer arithmetic operations with full control over signs and digit manipulation.

Internal Representation

- Uses a `private List<Integer> digits` to store digits of the number in reverse order — least significant digit (LSD) at index 0.
- This ordering simplifies arithmetic operations like carry/borrow propagation.
- Maintains a `public boolean isnegative` to represent the sign explicitly.
- Chose `List<Integer>` over `String` for better performance and flexibility during digit manipulation.

Accessors and Mutators

- `get_digits()`: Returns the internal digit list for inspection or low-level manipulation.
- `set_digits(List<Integer>)`: Allows safe replacement of the digit list, enabling transformations without exposing internals.

Constructors and Parsing

- **String constructor:**
 - Handles optional leading '-' to set `isnegative`.
 - Parses the numeric characters into reversed digits.
 - Ignores leading zeroes by trimming them after parsing.
- **Copy constructor:** Deep-copies another `AInteger`, ensuring mutability safety.
- **Empty constructor:** Initializes the object to represent the integer zero.
- **Static parser:** Provides a clean interface to parse a string and obtain an `AInteger`.

Arithmetic Operations

- All operations are implemented with full control over digit-wise calculation and sign logic.
- **Addition and Subtraction:**
 - Handles same-sign and different-sign logic explicitly.
 - Performs digit-wise carry and borrow tracking.
- **Multiplication:**
 - Uses standard long multiplication technique.
 - Result sign is derived from operand signs.
- **Division:**
 - Performs long division by repeated subtraction and digit estimation.

- Returns integer quotient only (no remainder or decimal part).

Formatting and Normalization

- `I_to_String()`: Converts the internal digit list to a human-readable string, adding ‘-’ prefix if negative.
- `removeLeadingZeroes()`: Normalizes internal representation by trimming leading zeroes to avoid redundancy.

AFloat

The `AFloat` class extends `AInteger` functionality to floating-point arithmetic with arbitrary precision by tracking decimal places through a `scale` parameter.

Internal Representation

- Each `AFloat` instance includes:
 - A private `AInteger` value storing all digits.
 - An integer `scale` indicating how many digits lie after the decimal point.
- Example: The float 1.23 is represented as:
 - `value = 123, scale = 2`

Constructors and Parsing

- **Default constructor:** Initializes the number as 0.0 with `scale = 1`.
- **String constructor:**
 - Parses optional negative sign and decimal point.
 - Counts decimal digits and updates scale accordingly.
 - Builds digit list in reverse, skipping the decimal point.
- **Copy constructor:** Clones another `AFloat` instance.
- **Helper constructor:** Creates a float from an `AInteger` and a scale.
- **Parser:** Static `parse(String)` method returns a parsed `AFloat`.

Arithmetic Operations

- **Addition & Subtraction**
 - Align scales of operands using `shift_deci`.
 - Perform underlying `AInteger` addition/subtraction.

- Return result with common scale, then normalize.
- **Multiplication**
 - Multiply internal `AInteger` values.
 - Final scale is the sum of operand scales.
- **Division**
 - Shifts numerator and denominator for precision handling.
 - Uses fixed decimal precision (default: 30 digits).
 - Returns normalized `AFloat` result.

Formatting and Normalization

- `F_to_String()`:
 - Assembles the output string with correct decimal positioning.
 - Inserts leading zeros when necessary for values < 1 .
- `removeTrailingZeros()`: Trims insignificant zeroes after the decimal.
- `removeLeadingZeros()`: Removes leading zeros before the decimal point.

Decimal Utility Function

- `shift_deci(int)` multiplies the number by 10^n to align decimal places during operations.

Class Diagram

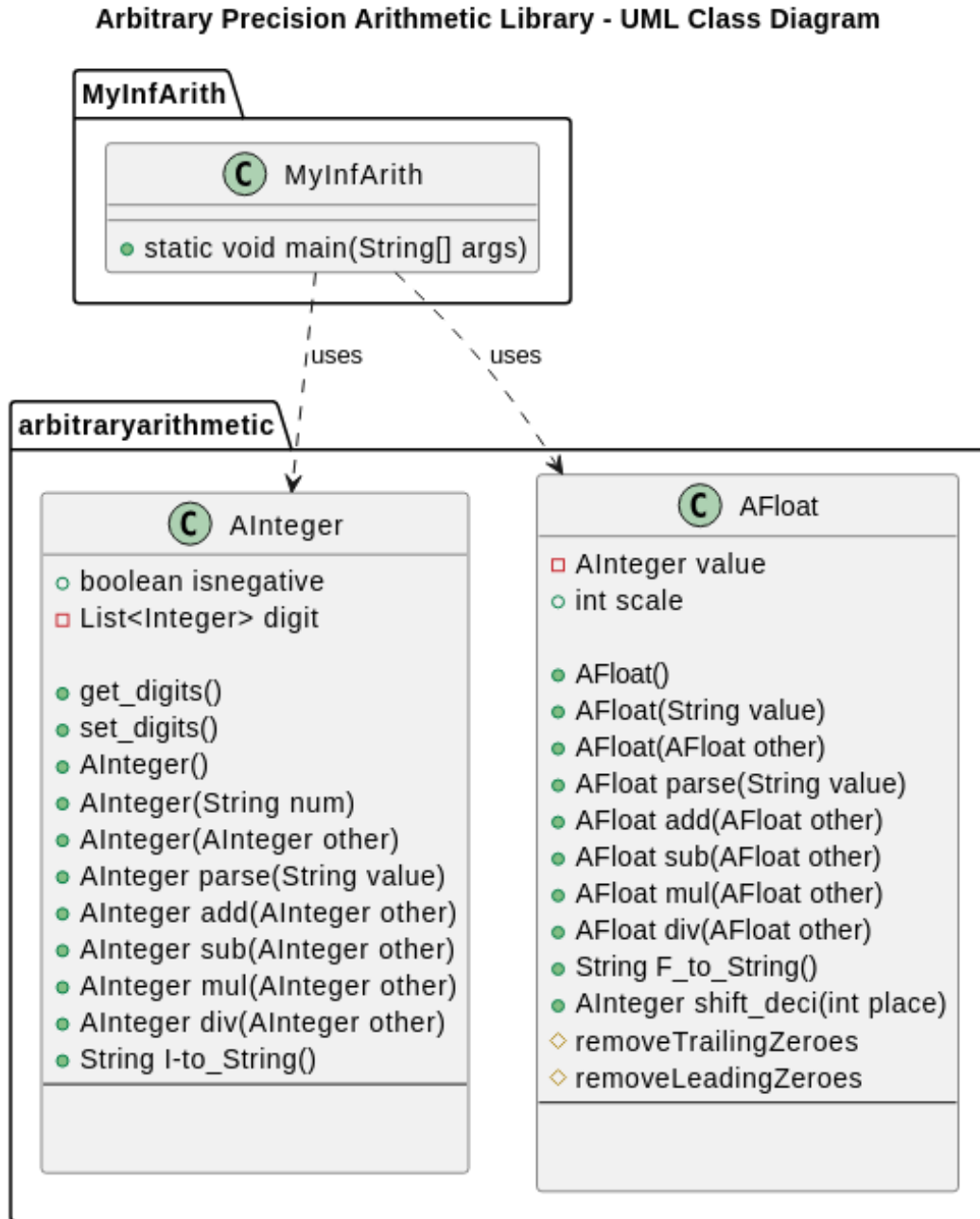


Figure 1: UML Diagram of the AInteger and AFloat Classes

3 User Guide (README Summary)

This section summarizes the README for end-users, outlining how to compile, execute, and integrate the library in different environments.

Compilation and Execution (Java CLI)

```
javac src/main/java/arbitraryarithmetic/*.java src/main/java/MyInfArith.java
java -cp src/main/java MyInfArith int add 123 456
```

Python CLI Interface

```
python run.py float mul 1.5 2.5
```

Using the Packaged JAR

```
java -cp target/aarithmetic.jar MyInfArith int sub 1000 250
```

Library Integration in Java Projects

```
import arbitraryarithmetic.*;

public class TestMain {
    public static void main(String[] args) {
        AInteger a = new AInteger("1234");
        AInteger b = new AInteger("4321");
        AInteger result = a.add(b);
        System.out.println(result); // Should print 5555
    }
}
```

Building with Maven

```
mvn clean install or mvn clean package
```

4 Git Commit History Snapshot

Including a visual summary of the git commit timeline. This helps in understanding the evolution and collaborative effort behind the project.

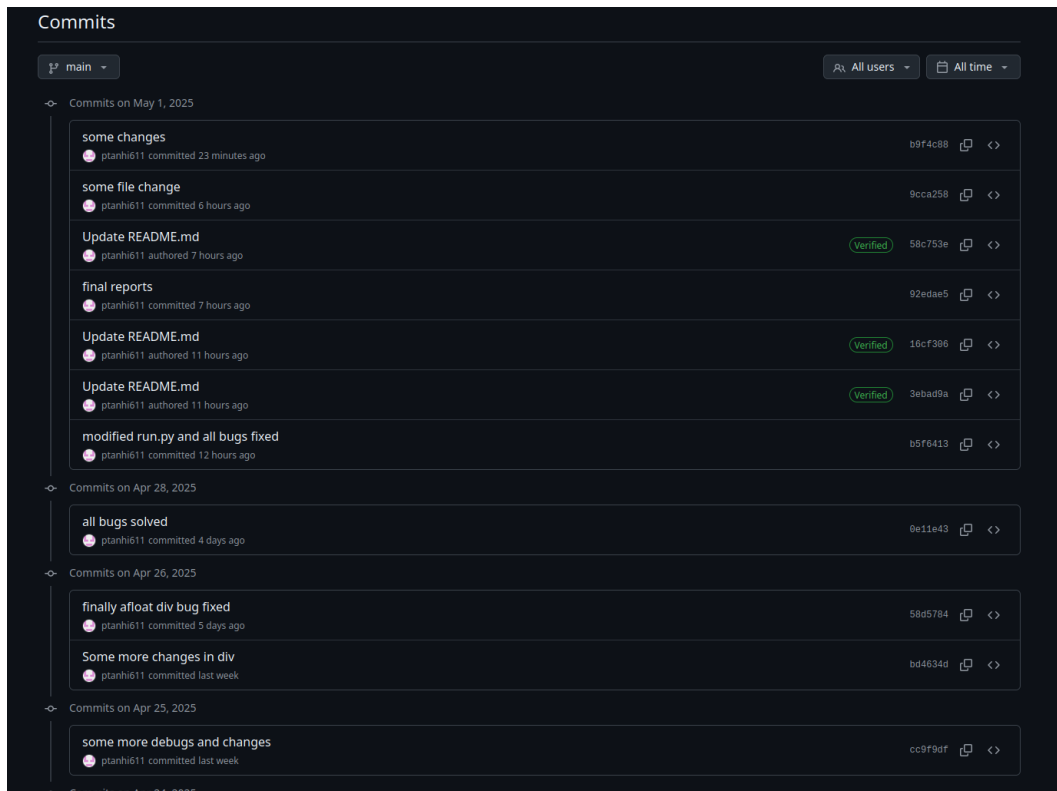


Figure 2: Snapshot of Git Commit History

5 Limitations

Despite the library's wide utility, it has the following limitations:

- *The Algorithm used for multiplication is simple school book method which is of complexity $O(n^2)$*
- *I thought of implementing some complex mathematical Algorithms for division and multiplication and also thought of using 10^8 as base instead of base 10 but the code was becoming too complicated for me, so I couldn't apply it.*
- *There is no stopage of printing of digits in float div until 30 digits even if the digits are repeating. Theres is round off for the last digit of the decimal (30^{th} place).*

6 Verification and Testing Approach

Robust verification was carried out through manual test cases designed to cover a comprehensive set of scenarios.

Test Scenarios

- Comparisons of positive vs. negative numbers

- Handling of extremely large numerical inputs
- Behavior with zero and identity elements
- Consistency in floating-point division
- Graceful handling of malformed or invalid inputs

Example Test Case

Input: `AInteger("999999999999") + AInteger("1")`

Expected Output: `AInteger("1000000000000")`

(and all the test cases given in the `.text` file)

I couldn't implement any automated testing framework but I tested it thoroughly covering every edge case and every functionality.

7 Key Learnings

This section reflects on the technical and collaborative takeaways from the project.

- Learned a lot of JAVA OOPS concepts.
- I Tried to implement few different Algorithms for optimization, but the code went too complex and I couldn't implement those fully in short time.
- Learned A lot about how to CLI to compile, run and build systems.
- Learned how to use git and push-pull remotely.